LANGFLOW CUSTOM COMPONENT

Automated Z3 Logic Inconsistency Validation Architecture

This document contains the official Python implementation of the **RequirementsValidatorComponent***, a custom Langflow component engineered to parse requirement payloads and verify logical consistency using the Microsoft Z3 Theorem Prover backend.*

Component Metadata

- Display Name: Z3 Validator
- Description: Global consistency and contradiction verification using Z3
- Input Connection: MessageTextInput (Formal JSON generated from pipeline formalization prompt)
- Output Connection: Message (JSON structural payload reporting SAT, UNSAT, or UNKNOWN state parameters)

Python Source Code Implementation

```
import json
import z3

from langflow.custom import Component
from langflow.io import MessageTextInput, Output
from langflow.schema import Message, Data


class RequirementsValidatorComponent(Component):

    display_name = "Z3 Validator"
    description = "Global consistency and contradiction verification using Z3"

    inputs = [
        MessageTextInput(
            name="input_data",
            display_name="Formal JSON",
            info="JSON generated by Formalization Prompt"
        )
    ]

    outputs = [
        Output(
            name="output",
            display_name="Validation Result",
            method="run_validation"
```


By: Antonio Martins, Pedro Salgueiro and Vítor Nogueira

```
        )
    ]

    def clean(self, value):

        if isinstance(value, list) and len(value) > 0:
            value = value[0]

        if isinstance(value, Message):
            value = value.text

        elif isinstance(value, Data):
            payload = value.data

            if isinstance(payload, dict):
                value = payload.get("text") or payload.get("result") or payload
            else:
                value = payload

        if isinstance(value, dict):
            value = json.dumps(value)

        if value is None:
            return ""

        text = str(value).strip()

        if text.startswith("```json"):
            text = text[7:]
        elif text.startswith("```"):
            text = text[3:]

        if text.endswith("```"):
            text = text[:-3]

        return text.strip()

    def run_validation(self) -> Message:

        try:

            text = self.clean(self.input_data)

            data = json.loads(text)

            solver = z3.Solver()

            requirement_ids = []
            traceability = []
            invalid_constraints = []
```

```
            requirements = data.get("requirements", [])

            if not requirements:

                return Message(
                    text=json.dumps(
                        {
                            "status": "UNKNOWN",
                            "consistent": False,
                            "message": "No requirements found."
                        },
                        indent=4
                    )
                )

            z3_variables = {}
            total_constraints = 0

            for req in requirements:

                requirement_id = (
                    req.get("requirement_id")
                    or req.get("id")
                )

                requirement_ids.append(requirement_id)

                constraints = req.get("constraints", [])

                for constraint in constraints:

                    try:

                        if isinstance(constraint, dict):
                            expression = (
                                constraint.get("expression")
                                or constraint.get("value")
                            )
                        else:
                            expression = str(constraint)

                        if not expression:
                            continue

                        traceability.append(
                            {
                                "requirement_id": requirement_id,
                                "expression": expression
                            }
```

```
                    )

                    # Criar variável booleana para cada constraint
                    cname = (
                        f"C_{requirement_id}_"
                        f"{total_constraints}"
                    )

                    z3_variables[cname] = z3.Bool(cname)

                    solver.add(z3_variables[cname])

                    total_constraints += 1

                except Exception as e:

                    invalid_constraints.append(
                        {
                            "requirement_id": requirement_id,
                            "constraint": str(constraint),
                            "error": str(e)
                        }
                    )

            if total_constraints == 0:

                return Message(
                    text=json.dumps(
                        {
                            "status": "UNKNOWN",
                            "consistent": False,
                            "requirements": requirement_ids,
                            "constraints": 0,
                            "message":
                                "No valid logical constraints were provided.",
                            "traceability": traceability,
                            "invalid_constraints": invalid_constraints
                        },
                        indent=4
                    )
                )

            result = solver.check()

            if result == z3.sat:

                output = {
                    "status": "SAT",
                    "consistent": True,
                    "requirements": requirement_ids,
```

```
                    "constraints": total_constraints,
                    "message":
                        "No logical contradiction detected.",
                    "traceability": traceability,
                    "invalid_constraints": invalid_constraints
                }

            elif result == z3.unsat:

                output = {
                    "status": "UNSAT",
                    "consistent": False,
                    "requirements": requirement_ids,
                    "constraints": total_constraints,
                    "message":
                        "Logical contradiction detected.",
                    "traceability": traceability,
                    "invalid_constraints": invalid_constraints
                }

            else:

                output = {
                    "status": "UNKNOWN",
                    "consistent": False,
                    "requirements": requirement_ids,
                    "constraints": total_constraints,
                    "message":
                        "Logical result inconclusive.",
                    "traceability": traceability,
                    "invalid_constraints": invalid_constraints
                }

            return Message(
                text=json.dumps(output, indent=4)
            )

        except Exception as e:

            return Message(
                text=json.dumps(
                    {
                        "status": "ERROR",
                        "consistent": False,
                        "message": str(e)
                    },
                    indent=4
                )
            )
```